Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load the dataset
```

```
df = pd.read_csv(
    'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titani
)

# Remove rows with missing Age values
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data into training and testing sets
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)

# Create and train the model
model_reg = LinearRegression()
model_reg.fit(X_train_reg, y_train_reg)

# Make predictions
y_pred_reg = model_reg.predict(X_test_reg)

# Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)

print("Mean Squared Error:", mse)
print("First 5 actual fares:", y_test_reg.head().tolist())
print("First 5 predicted fares:", y_pred_reg[:5])
```

```
Mean Squared Error: 3364.922912856981
First 5 actual fares: [13.0, 18.75, 26.0, 69.3, 6.45]
First 5 predicted fares: [38.36041481 55.40181133 44.04088032 84.52296409 -0.3
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Create the model instance
lr_model = LinearRegression()

# 2. Train the model
lr_model.fit(X_train_reg, y_train_reg)

# 3. Make predictions on the test data
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)

print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model's pr
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model's predictions are off by a
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# Prepare data for classification
# Encode Sex: male = 0, female = 1
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Remove rows with missing values in the required columns
df.dropna(subset=['Age', 'Pclass', 'Sex', 'Survived'], inplace=True)

# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]
```

```
# Split the data into training and testing sets
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls,
    y_cls,
    test_size=0.2,
    random_state=42
)

# Create the Logistic Regression model
log_model = LogisticRegression(max_iter=1000)

# Train the model
log_model.fit(X_train_cls, y_train_cls)

# Make predictions
y_pred_cls = log_model.predict(X_test_cls)

# Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)

print(f"Logistic Regression Accuracy: {accuracy:.2%}")
```

```
Logistic Regression Accuracy: 74.83%
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Create the model instance
log_model = LogisticRegression(max_iter=1000)

# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)

print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent? 3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?[ENTER YOUR ANSWERS HERE]**

1. What is the key difference between regression and classification?

Regression is used to predict a number, like how much a Titanic passenger paid for their fare. Classification is used to predict a category, like whether a passenger survived or not.

2. What do the numbers in `lr_model.coef_` represent?

The numbers show how each feature affects the predicted fare. In our model, they tell us how Age and passenger class affect the fare when the other feature stays the same.

3. Why did we use `mean_squared_error` for regression and `accuracy_score` for classification? Why wouldn't accuracy be good for predicting fares?

We used Mean Squared Error because it measures the difference between the actual fares and the fares predicted by the model. We used accuracy for classification because it tells us how many predictions were correct. Accuracy would not work well for predicting fares because fares are numbers, not categories. Mean Squared Error helps us see how far off the predicted fares are from the actual fares.